# JAVA PROGRAMMING EXERCISE SERIES

*Real-World Case Study Exercises: Beginner to Professional*

5 Levels • 50 Exercises • Industry Scenarios

*Designed for University Students, Coding Bootcamps & Self-Learners*

| Level 1 | Level 2 | Level 3 | Level 4 | Level 5 |
|---|---|---|---|---|
| BEGINNER | ELEMENTARY | INTERMEDIATE | ADVANCED | PROFESSIONAL |

# About This Exercise Series

This Java Programming Exercise Series is a structured, real-world training roadmap designed to take learners from absolute beginner to professional-grade Java developer. Unlike traditional textbook exercises, every exercise in this series is grounded in realistic industry scenarios drawn from sectors including healthcare, banking, retail, hospitality, education, logistics, and smart city infrastructure.

The series comprises 50 exercises distributed across 5 progressive levels, each containing 10 case-study-based problems. Exercises are carefully sequenced so that each new concept is introduced in context and reinforced through subsequent exercises. By the time a learner reaches Level 5, they will have built complete, multi-tier Java applications incorporating database connectivity, REST API integration, design patterns, concurrent programming, graphical user interfaces, and comprehensive unit testing.

Each exercise follows a consistent structure — providing scenario background, a clear problem statement, measurable learning objectives, input/output specifications, constraints, suggested Java concepts, a solution approach guide, and an optional challenge extension for advanced learners.

## Learning Roadmap Overview

| Level | Title | Key Concepts | Exercises | Est. Time |
|---|---|---|---|---|
| Level 1 | Beginner | Variables, I/O, Operators, Loops, Arrays, Strings | 10 | 5–10 hrs |
| Level 2 | Elementary | Methods, String Methods, 2D Arrays, Static Members | 10 | 10–15 hrs |
| Level 3 | Intermediate | OOP: Classes, Inheritance, Interfaces, Polymorphism | 10 | 20–30 hrs |
| Level 4 | Advanced | Exceptions, Collections, File I/O, JDBC, Threads | 10 | 30–40 hrs |
| Level 5 | Professional | Design Patterns, REST API, Testing, JavaFX | 10 | 40–60 hrs |

## Industry Scenarios Used

| Organisation | System / Domain |
|---|---|
| **City Public Library** | Library Management System – cataloguing, borrowing, member management |
| **Sunrise General Hospital** | Hospital Management System – patient records, appointments, blood bank |
| **National Savings Bank** | Banking System – accounts, transactions, transfers, fraud detection |
| **ShopEase Online Platform** | E-Commerce – product catalogue, cart, orders, analytics dashboard |
| **Serenity Boutique Hotel** | Hotel Reservation – room booking, services, billing, occupancy |
| **Westbrook University** | Student Information System – enrolment, grades, transcripts, GPA ranking |
| **TechSoft Solutions** | Employee Payroll – pay types, deductions, tax, payslip generation |
| **QuickBite Food Delivery** | Food Delivery App – ordering, tracking, pricing, concurrent processing |

| | |
|---|---|
| **MegaMart Inventory** | Inventory Management – stock tracking, reorder, expiry, supplier management |
| **SmartPark City Parking** | Smart Parking System – slot allocation, ticketing, payment, analytics |

## How to Use This Series

✓ Work sequentially: each level builds on concepts introduced in previous levels. Skipping levels is not recommended.

✓ Read the Scenario Background carefully before starting – understanding the real-world context makes the solution more meaningful.

✓ Attempt the Problem Statement independently before consulting the Solution Approach guide.

✓ Complete the Challenge Extension once your base solution works to push your skills further.

✓ In Level 3 and above, draw a class diagram before writing any code. Design first, code second.

✓ For Level 4–5 exercises, set up a proper Maven or Gradle project with all dependencies declared.

✓ Track your progress: time yourself against the estimated completion time to measure improvement.

# LEVEL 1
# BEGINNER

*Console-Based Fundamentals*

Variables • Data Types • I/O • Operators • Conditionals • Loops • Strings

## Exercise 1.1 Library Member Registration

*Scenario: City Public Library*

⏱ 30–45 min
🎯 **Easy**

| | |
|---|---|
| **BACK- GROUND** | The City Public Library wants to automate its member registration process. Currently, staff manually fill paper forms for each new member. The library needs a simple console program to collect and display member details. |
| **PROBLEM** | Write a Java program that collects new member details from the console, calculates the membership expiry date (1 year from today), and prints a formatted membership card summary. |
| **OBJECTIVES** | • Declare and use primitive variables (int, double, String)<br>• Use Scanner for console input<br>• Perform basic arithmetic with dates<br>• Format and print structured output |
| **INPUT REQUIREMENTS**<br>• Member Name (String)<br>• Age (int)<br>• Contact Number (String)<br>• Email Address (String)<br>• Membership Type: Standard/Premium (String) | **EXPECTED OUTPUT**<br>Formatted membership card showing Member ID (auto-generated), name, age, contact, email, membership type, registration date, and expiry date. |
| **CON- STRAINTS** | • Age must be between 5 and 100<br>• Membership ID auto-generated as LIB + year + 3-digit sequence |
| **JAVA CONCEPTS** | • Variables & Data Types<br>• Scanner (System.in)<br>• System.out.printf()<br>• String concatenation<br>• java.time.LocalDate |
| **SOLUTION APPROACH** | Declare variables for each field, use Scanner to read inputs, apply LocalDate.now() for date logic, and use printf() with format specifiers to produce the membership card. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add input validation using if-else to reject invalid ages and print an error message instead of the card. |

## Exercise 1.2 Supermarket Bill Calculator

*Scenario: FreshMart Supermarket*

⏱ 30–45 min
Easy

| | |
|---|---|
| BACK- GROUND | FreshMart Supermarket needs a simple billing console program for its cashiers to quickly calculate customer totals, apply discounts, and compute change. |
| PROBLEM | Create a Java program that accepts up to 5 items (name, price, quantity), calculates the subtotal, applies a 10% discount if total exceeds LKR 5,000, adds 8% VAT, and prints the final bill. |
| OBJECTIVES | • Use arithmetic operators (+, -, *, /)<br>• Apply conditional logic for discounts<br>• Format currency output using printf()<br>• Use constants for tax and discount rates |
| INPUT REQUIREMENTS | • Number of items (max 5)<br>• For each item: name, unit price, quantity |
| EXPECTED OUTPUT | Itemised bill showing each item total, subtotal, discount amount (if applicable), VAT amount, and grand total with change calculation. |
| CON- STRAINTS | • Maximum 5 items per transaction<br>• Prices must be positive values<br>• VAT rate = 8%, Discount = 10% if subtotal > 5000 |
| JAVA CONCEPTS | • Arithmetic operators<br>• final constants<br>• Conditional statements<br>• printf() formatting |
| SOLUTION APPROACH | Use a loop to collect item details, accumulate totals with running sum, apply conditional discount logic, compute VAT on discounted amount, and display results using printf with %.2f. |
| CHALLENGE EXTENSION | Add a loyalty points system: 1 point per LKR 100 spent. Display total points earned after payment. |

## Exercise 1.3 Hospital Patient Admission Checker

⏱ 45–60 min
Easy

*Scenario: Sunrise General Hospital*

| | |
|---|---|
| BACK- GROUND | Sunrise General Hospital's front desk needs a quick-check tool to determine which ward a newly arrived patient should be assigned to, based on their reported symptoms and vital signs. |
| PROBLEM | Write a Java program that accepts patient name, age, temperature (°C), blood pressure (systolic), and primary symptom. Based on the inputs, determine and print the recommended ward and priority level. |
| OBJECTIVES | • Use if-else-if chains for multi-condition logic<br>• Compare numeric ranges<br>• Use String methods (equalsIgnoreCase)<br>• Understand logical operators (&& and \|\|) |
| INPUT REQUIREMENTS<br>• Patient Name<br>• Age<br>• Body Temperature (°C)<br>• Systolic Blood Pressure (mmHg)<br>• Primary Symptom (fever/chest pain/fracture/other) | EXPECTED OUTPUT<br>Admission report with patient details, determined ward (ICU/Emergency/General/Paediatric), and priority level (Critical/High/Normal). |
| CON- STRAINTS | • Temp > 39.5°C → High fever; BP > 180 → Hypertensive crisis; chest pain → Emergency; Age < 12 → Paediatric |
| JAVA CONCEPTS | • if-else-if ladder<br>• Logical operators<br>• String.equalsIgnoreCase()<br>• Relational operators |
| SOLUTION APPROACH | Use nested if-else conditions checking each vital sign threshold and symptom. Assign ward and priority based on matched conditions, then print a formatted report. |
| CHALLENGE EXTENSION | Add a switch statement to select the doctor on duty per ward from a pre-defined list. |

## Exercise 1.4 ATM Cash Withdrawal Simulator

45–60 min

**Easy**

*Scenario: National Savings Bank*

| | |
|---|---|
| BACK- GROUND | National Savings Bank wants a console-based ATM simulator for training new tellers. It should simulate PIN verification, balance enquiry, and cash withdrawal with daily limit enforcement. |

| | |
|---|---|
| **? PROBLEM** | Create a Java program simulating an ATM session. The program should verify a 4-digit PIN (hardcoded), show balance, allow withdrawal (with daily limit of LKR 50,000), and print a transaction receipt. |
| **OBJECTIVES** | • Use while loops for retry logic<br>• Apply nested conditionals<br>• Maintain a running balance using variables<br>• Understand basic control flow |
| **INPUT REQUIREMENTS**<br>• PIN (4-digit integer)<br>• Withdrawal amount | **EXPECTED OUTPUT**<br>ATM screen output: welcome, balance display, withdrawal confirmation or error, and receipt with transaction ID and remaining balance. |
| **⚠ CON- STRAINTS** | • 3 PIN attempts max before card lockout<br>• Daily withdrawal limit: LKR 50,000<br>• Minimum withdrawal: LKR 500<br>• Multiples of 500 only |
| **JAVA CONCEPTS** | • while loop<br>• if-else<br>• int arithmetic<br>• String formatting<br>• break statement |
| **SOLUTION APPROACH** | Use a while loop allowing up to 3 PIN attempts. On correct PIN, enter a do-while menu loop. Check amount against balance and daily limit before processing withdrawal. |
| **CHALLENGE EXTENSION** | Add deposit functionality and a mini-statement showing the last 3 transactions using three separate variables. |

## Exercise 1.5 Student Grade Calculator

*Scenario: Westbrook University*

⏱ 30–45 min
**Easy**

| | |
|---|---|
| **BACK- GROUND** | Westbrook University's examination department needs a tool to calculate a student's final grade from subject marks, determine GPA, and produce an official grade report. |
| **? PROBLEM** | Write a Java program that accepts marks for 5 subjects, calculates the average, assigns a letter grade (A/B/C/D/F) and GPA (4.0 scale), and prints an academic report card. |
| **OBJECTIVES** | • Use loops to process repeated inputs<br>• Apply switch or if-else for grade classification<br>• Compute averages and percentages<br>• Format tabular output |

| | |
|---|---|
| **INPUT REQUIREMENTS**<br>• Student Name, ID, Semester<br>• Mark for each of 5 subjects (0–100) | **EXPECTED OUTPUT**<br>Report card showing each subject mark and grade, cumulative GPA, overall letter grade, and pass/fail status (pass if average ≥ 50). |
| **CON- STRAINTS** | • Marks must be 0–100<br>• GPA: A=4.0(90+), B=3.0(75–89), C=2.0(60–74), D=1.0(50–59), F=0(below 50) |
| **JAVA CONCEPTS** | • for loop<br>• switch statement<br>• Arithmetic<br>• printf()<br>• boolean flags |
| **SOLUTION APPROACH** | Loop 5 times to read marks, accumulate sum, compute average, use if-else or switch to assign grade and GPA, then print results in a formatted table. |
| **CHALLENGE EXTENSION** | Find and display the highest and lowest marks along with their subject names. |

## Exercise 1.6 Hotel Room Rate Calculator

*Scenario: Serenity Boutique Hotel*

45 min
**Easy**

| | |
|---|---|
| **BACK- GROUND** | Serenity Boutique Hotel needs a quick-quote console tool that calculates the total cost of a guest's stay based on room type, number of nights, season, and additional services. |
| **PROBLEM** | Create a Java program that lets a guest select a room type, specify check-in/check-out (number of nights), choose add-ons (breakfast, airport transfer, spa), and compute the total bill. |
| **OBJECTIVES** | • Use switch for menu-based room selection<br>• Accumulate costs with boolean add-ons<br>• Apply seasonal pricing logic<br>• Format currency output |
| **INPUT REQUIREMENTS**<br>• Room type: Standard/Deluxe/Suite (1/2/3)<br>• Number of nights<br>• Season: Peak/Off-Peak (1/2)<br>• Add-ons: Breakfast (Y/N), Transfer (Y/N), Spa (Y/N) | **EXPECTED OUTPUT**<br>Booking quote showing room rate per night, number of nights, add-on costs, seasonal surcharge (20% in peak), and total amount due. |

| | |
|---|---|
| ⚠ CON- STRAINTS | • Standard: LKR 8,000/night, Deluxe: LKR 15,000/night, Suite: LKR 28,000/night<br>• Peak season surcharge: 20% |
| JAVA CONCEPTS | • switch statement<br>• boolean variables<br>• Arithmetic operators<br>• Formatted output |
| SOLUTION APPROACH | Use switch for room selection, if-else for season, and boolean checks for add-ons. Accumulate each cost component and display an itemised breakdown. |
| CHALLENGE EXTENSION | Apply a 5% loyalty discount if the guest enters a valid loyalty number (any 8-character alphanumeric code). |

## Exercise 1.7 Employee Work Hours Tracker

*Scenario: TechSoft Solutions Pvt Ltd*

⏱ 45–60 min
Easy

| | |
|---|---|
| BACK- GROUND | TechSoft Solutions needs a simple payroll assistant to track daily work hours for a single employee over a work week and calculate gross pay including overtime. |
| PROBLEM | Write a Java program that accepts an employee's name, hourly rate, and daily hours worked for Monday to Saturday. Calculate total hours, regular pay, overtime pay (>8h/day at 1.5x), and weekly gross. |
| OBJECTIVES | • Use arrays to store daily data<br>• Use loops for computation<br>• Apply conditional logic for overtime<br>• Compute cumulative totals |
| INPUT REQUIREMENTS<br>• Employee Name<br>• Hourly Rate (LKR)<br>• Hours worked: Monday through Saturday (6 values) | EXPECTED OUTPUT<br>Weekly pay summary: daily breakdown showing regular/overtime hours and pay per day, total regular hours, total OT hours, and gross weekly pay. |
| ⚠ CON- STRAINTS | • Standard hours per day = 8; anything above is overtime<br>• Max 16 hours per day<br>• Hourly rate must be > 0 |
| JAVA CONCEPTS | • Arrays (int[])<br>• for loop<br>• Conditional (if-else)<br>• Accumulator pattern |

| | |
|---|---|
| SOLUTION APPROACH | Store hours in a double array. Loop through each day: if hours > 8, split into regular and OT hours. Multiply by rate and 1.5x rate respectively. Sum all components. |
| CHALLENGE EXTENSION | Deduct EPF (8% employee) and ETF (3% employer contribution shown separately) and display net pay. |

## Exercise 1.8 Food Order Validator

*Scenario: QuickBite Food Delivery App*

30–45 min

Easy

| | |
|---|---|
| BACK- GROUND | QuickBite's backend team needs a console-based order validation module to ensure incoming food orders meet business rules before being forwarded to the restaurant. |
| PROBLEM | Write a Java program that accepts a customer order (customer name, delivery address, number of items 1–10, each with name and price) and validates it against business rules before confirming. |
| OBJECTIVES | • Use Scanner for structured multi-field input<br>• Apply validation logic with if-else<br>• Compute totals and apply minimum order rule<br>• Use String validation methods |
| INPUT REQUIREMENTS | • Customer name, phone (10 digits), address<br>• Number of items (1–10)<br>• Each item: name and price |
| EXPECTED OUTPUT | Order validation report: ACCEPTED or REJECTED with reasons if rejected. If accepted, print order summary with subtotal, delivery fee (LKR 150), and total. |
| CON- STRAINTS | • Minimum order value: LKR 500 (excluding delivery)<br>• Phone must be 10 digits<br>• Item price must be positive<br>• Address cannot be empty |
| JAVA CONCEPTS | • String.length()<br>• String.isEmpty()<br>• if-else validation chain<br>• for loop<br>• double accumulation |
| SOLUTION APPROACH | Read all inputs first, then apply each validation rule with descriptive error messages. Use a boolean flag to track overall validity. Only print confirmation if all checks pass. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add a promo code field. If code = 'QUICKBITE20', apply 20% discount to subtotal. |

## Exercise 1.9 Parking Fee Calculator

*Scenario: SmartPark City Parking System*

⏱ 30–40 min
**Easy**

| | |
|---|---|
| BACK- GROUND | SmartPark operates automated parking lots. Their ticketing system needs a simple program to calculate parking fees based on vehicle type, entry time, and exit time. |
| PROBLEM | Create a Java program that accepts vehicle type, entry hour, and exit hour (24-hour format), calculates duration, determines the applicable rate, and prints a parking receipt. |
| OBJECTIVES | • Perform integer arithmetic for time calculation<br>• Use switch for vehicle type selection<br>• Apply tiered pricing logic<br>• Generate a formatted receipt |

**INPUT REQUIREMENTS**

- Vehicle Type: Bike/Car/Van/Lorry (1–4)
- Entry hour (0–23)
- Exit hour (0–23)
- Vehicle number plate

**EXPECTED OUTPUT**

Parking receipt: vehicle number, type, entry/exit time, duration in hours, rate per hour, and total fee. Include 'NIGHT RATE' label if any hour is between 22:00 and 06:00.

| | |
|---|---|
| CON- STRAINTS | • Night rate (10 PM–6 AM): 1.5x normal rate<br>• Minimum charge: 1 hour<br>• Bike: LKR 30/h, Car: LKR 80/h, Van: LKR 150/h, Lorry: LKR 250/h |
| JAVA CONCEPTS | • switch statement<br>• int arithmetic<br>• Math.max()<br>• Conditional for night rate<br>• printf() |
| SOLUTION APPROACH | Calculate duration with exit-entry (handle overnight by adding 24 if exit < entry). Use switch for base rate. Check if any hour falls in night-rate window and apply 1.5x accordingly. |
| CHALLENGE EXTENSION | Add a monthly pass check: if the user enters pass number 'SMART-XXXX' where X is any digit, waive the fee and print 'PASS HOLDER'. |

## Exercise 1.10 Inventory Stock Checker

*Scenario: MegaMart Inventory Control*

45–60 min

**Easy**

| | |
|---|---|
| **BACK- GROUND** | MegaMart's warehouse manager uses a manual process to check if stock levels need replenishment. The company wants a simple console program to automate daily stock checks for up to 10 products. |
| **PROBLEM** | Write a Java program that accepts details for up to 10 products (name, current stock, minimum threshold, unit price), identifies which products need reordering, and prints a restocking report. |
| **OBJECTIVES** | • Use parallel arrays for related data<br>• Apply loops and conditionals together<br>• Compute reorder quantities<br>• Sort and display results |
| **INPUT REQUIREMENTS**<br>• Number of products (1–10)<br>• For each: product name, current stock, minimum threshold, unit price | **EXPECTED OUTPUT**<br>Stock status report: table of all products with status (OK/LOW/CRITICAL). Summary showing count of items needing reorder, estimated reorder cost (order to 2x threshold). |
| **CON- STRAINTS** | • CRITICAL: stock < 50% of threshold; LOW: stock < threshold; OK: stock >= threshold<br>• Reorder quantity = 2 × threshold − current stock |
| **JAVA CONCEPTS** | • Parallel String[] and int[] arrays<br>• for loop<br>• if-else-if<br>• String.format()<br>• Accumulator |
| **SOLUTION APPROACH** | Use four parallel arrays for name, stock, threshold, and price. Loop through each product, compare stock to threshold, classify status, and compute reorder cost. Print a formatted table. |
| **CHALLENGE EXTENSION** | Sort the array by reorder priority (CRITICAL first, then LOW, then OK) using a simple bubble sort before printing. |

# LEVEL 2
# ELEMENTARY

*Methods, Strings & Basic Arrays*

Methods • String Methods • Multi-dim Arrays • Static Members • Basic OOP Intro

## Exercise 2.1 Library Book Search Engine

*Scenario: City Public Library*

⏱ 60–75 min
🎯 **Easy–Medium**

**BACK- GROUND**

The library's catalogue has grown to hundreds of books. Staff need a method-driven search tool that can find books by title keyword, author name, or ISBN without needing a database.

**PROBLEM**

Build a Java program with static methods to search, display, and sort a hardcoded library catalogue (10 books stored in parallel arrays). Implement search by title keyword, by author, and by genre.

**OBJECTIVES**

- Define and call static methods
- Pass arrays as parameters
- Use String methods (contains, toLowerCase, startsWith)
- Return values from methods

**INPUT REQUIREMENTS**

- Search type selection (1=Title, 2=Author, 3=Genre)
- Search keyword

**EXPECTED OUTPUT**

Formatted list of matching books: Book ID, Title, Author, Genre, Year, and Availability. 'No results found' if no match.

**CON- STRAINTS**

- Case-insensitive search
- Partial match allowed for title/author
- Exact match for genre

**JAVA CONCEPTS**

- static methods
- String.contains()
- String.toLowerCase()
- Array traversal
- Method return types

**SOLUTION APPROACH**

Define separate static search methods each taking the arrays and query string. Loop through arrays applying appropriate String method. Collect results and print from a display method.

**CHALLENGE EXTENSION**

Add a borrow() method that marks a book as unavailable. Track borrows in a boolean[] array.

## Exercise 2.2 Hospital Appointment Scheduler

*Scenario: Sunrise General Hospital*

⏱ 60–90 min
🎯 **Medium**

| | |
|---|---|
| 📖 BACK- GROUND | The outpatient department manages daily appointments manually. A simple console scheduler needs to let staff book, view, and cancel appointments for doctors using arrays and methods. |
| ❓ PROBLEM | Create a method-based appointment system for 3 doctors, each with 5 daily slots. Implement book, cancel, view all, and view by doctor functionality using static methods. |
| 🎓 OBJECTIVES | • Use 2D arrays to model doctor-slot grid<br>• Define methods with multiple parameters<br>• Validate slot availability before booking<br>• Use String formatting for schedule display |
| 📥 INPUT REQUIREMENTS | • Menu selection<br>• Doctor ID (1–3)<br>• Slot number (1–5)<br>• Patient name and contact |
| 📤 EXPECTED OUTPUT | Visual schedule grid showing each doctor's slots (AVAILABLE / patient name). Booking confirmation or error messages. |
| ⚠ CON- STRAINTS | • Cannot double-book a slot<br>• Cancellation requires correct doctor ID + slot number<br>• 5 slots per doctor per day |
| 💡 JAVA CONCEPTS | • 2D arrays (String[][])<br>• static methods<br>• Nested loops<br>• String.format()<br>• Menu-driven while loop |
| 🔧 SOLUTION APPROACH | Model the schedule as a 3×5 String[][] array (null = available). Each menu option calls a dedicated method. Booking sets the slot; cancellation nullifies it. Display iterates the 2D array. |
| 🚀 CHALLENGE EXTENSION | Add a search method to find which slot a patient is booked in by name. |

## Exercise 2.3 Bank Account Transaction Logger

*Scenario: National Savings Bank*

⏱ 75–90 min
🎯 **Medium**

| | |
|---|---|
| BACK- GROUND | The bank's core system needs a transaction log module that records deposits and withdrawals for a single account session and produces a mini-statement with running balance. |
| PROBLEM | Design a static-method-based transaction logger. Implement deposit(), withdraw(), getBalance(), printStatement(), and findLargestTransaction() methods working on shared static arrays. |
| OBJECTIVES | • Use static class-level arrays and variables<br>• Build methods with input validation<br>• Generate formatted account statements<br>• Find min/max in an array |
| INPUT REQUIREMENTS<br>• Transaction type (D/W/Q for deposit/withdrawal/quit)<br>• Transaction amount | EXPECTED OUTPUT<br>Live balance update after each transaction. Final mini-statement table: date-time, type, amount, running balance. Summary: total deposits, total withdrawals, largest single transaction. |
| CON- STRAINTS | • Insufficient funds: reject withdrawal gracefully<br>• Max 20 transactions per session<br>• Minimum deposit: LKR 100 |
| JAVA CONCEPTS | • static arrays and variables<br>• Methods with validation<br>• Parallel arrays for log<br>• String formatting<br>• Running total pattern |
| SOLUTION APPROACH | Use static arrays for amounts, types, and timestamps. Each method validates its own preconditions. Statement method loops through the transaction arrays producing a formatted table. |
| CHALLENGE EXTENSION | Implement a searchByAmount() method that returns all transactions above a given threshold. |

## Exercise 2.4 Online Store Product Catalogue

*Scenario: ShopEase Online Platform*

60–75 min

**Medium**

| | |
|---|---|
| BACK- GROUND | ShopEase needs a command-line product catalogue viewer where customers can browse products, filter by price range, and add items to a simple cart before checking out. |

| | |
|---|---|
| ? PROBLEM | Build a method-based product catalogue with at least 15 products. Implement filterByPrice(), filterByCategory(), addToCart(), removeFromCart(), and printCart() static methods. |
| OBJECTIVES | • Design multiple co-operating static methods<br>• Use parallel arrays for product data<br>• Implement a cart using a sub-array<br>• Compute cart totals with methods |
| INPUT REQUIREMENTS<br>• Menu choice<br>• Filter bounds (min/max price)<br>• Product ID to add/remove | EXPECTED OUTPUT<br>Formatted product list per filter. Cart contents showing product names, prices, quantities, and total. Checkout receipt. |
| ⚠ CON- STRAINTS | • Cart limited to 10 items<br>• Cannot add out-of-stock items (boolean[] inStock)<br>• Quantity per item: max 5 |
| JAVA CONCEPTS | • static methods<br>• boolean arrays<br>• Array search<br>• Sub-array for cart<br>• Accumulator methods |
| SOLUTION APPROACH | Separate each concern into its own method. Filter methods create result sub-arrays. Cart methods use a separate int[] for quantities indexed by product ID. Total method sums cart contents. |
| CHALLENGE EXTENSION | Add a sortByPrice() method using bubble sort and let the user toggle ascending/descending. |

## Exercise 2.5 Student GPA Rank Generator

*Scenario: Westbrook University*

⏱ 60–75 min
**Medium**

| | |
|---|---|
| BACK- GROUND | The university registrar needs an end-of-semester ranking report that sorts students by GPA, identifies honours students, and produces class statistics. |
| ? PROBLEM | Write a Java program using methods to manage a class of 20 students. Implement calculateGPA(), rankStudents() (selection sort by GPA), countHonours(), getClassAverage(), and printRanklist() methods. |
| OBJECTIVES | • Implement a sorting algorithm as a method |

| | |
|---|---|
| | • Compute derived values (GPA) from raw marks<br>• Use methods to encapsulate statistical calculations<br>• Generate formatted rank lists |
| **INPUT REQUIREMENTS**<br>• Student name and ID for each of 20 students<br>• 5 subject marks per student | **EXPECTED OUTPUT**<br>Ranked class list: Rank, Student ID, Name, GPA, Honours status (GPA ≥ 3.5). Class statistics: average GPA, highest/lowest GPA, honours count. |
| **⚠ CON- STRAINTS** | • GPA scale: A=4, B=3, C=2, D=1, F=0<br>• Alphabetical tie-breaking for equal GPA<br>• At least 20 students |
| **JAVA CONCEPTS** | • Multi-dim arrays<br>• Selection sort<br>• Static calculation methods<br>• String comparison<br>• Formatted tables |
| **SOLUTION APPROACH** | Store all data in a 2D double[][] for marks. A separate method calculates and stores GPAs. rankStudents() implements selection sort on the GPA array, simultaneously reordering the name array. |
| **CHALLENGE EXTENSION** | Add a method to identify the top 3 students in each subject individually. |

## Exercise 2.6 Restaurant Menu and Ordering

*Scenario: QuickBite Food Delivery App*

⏱ 60 min
**Easy–Medium**

| | |
|---|---|
| **BACK- GROUND** | QuickBite needs a console-based menu ordering system for restaurant partners. Staff should be able to browse the menu by category, place orders, and modify them before confirmation. |
| **? PROBLEM** | Build a method-based console ordering system for a restaurant with 20 menu items across 4 categories. Implement displayMenu(), searchItem(), placeOrder(), modifyOrder(), and generateBill(). |
| **OBJECTIVES** | • Organise data using category-indexed arrays<br>• Implement item search by name substring<br>• Build a modifiable order session<br>• Generate an itemised bill using methods |
| **INPUT REQUIREMENTS**<br>• Category selection | **EXPECTED OUTPUT** |

| | |
|---|---|
| • Item ID<br>• Quantity<br>• Modification commands (add/remove/change quantity) | Menu display per category. Running order summary. Final bill with itemised list, subtotal, 10% service charge, and total. |
| ⚠ CON- STRAINTS | • Max 15 order lines<br>• Minimum order: LKR 300<br>• Service charge: 10% |
| JAVA CONCEPTS | • Parallel String/double/int arrays<br>• Static methods<br>• String.contains()<br>• Order array management<br>• Formatted output |
| SOLUTION APPROACH | Use category arrays to group items. Search scans all items for keyword. Order stored in item-ID and quantity arrays. Bill method sums quantities × prices then applies service charge. |
| CHALLENGE EXTENSION | Add an estimatedDeliveryTime() method that returns '30–45 mins' for <5 items and '45–60 mins' otherwise. |

## Exercise 2.7 Payroll Slip Generator

*Scenario: TechSoft Solutions Pvt Ltd*

⏱ 75–90 min
**Medium**

| | |
|---|---|
| BACK- GROUND | TechSoft HR wants a monthly payroll slip generator that computes gross pay, all statutory deductions, and net pay for employees, producing a formatted pay slip. |
| ? PROBLEM | Create a Java program with dedicated static methods for each payroll component: computeGross(), computeEPF(), computeETF(), computeTax(), computeNet(), and printPaySlip(). Process 5 employees. |
| OBJECTIVES | • Break a complex problem into single-responsibility methods<br>• Apply progressive tax brackets with methods<br>• Return multiple values using arrays<br>• Format a professional pay slip output |
| **INPUT REQUIREMENTS**<br>• For each employee: name, employee ID, base salary, overtime hours, allowances | **EXPECTED OUTPUT**<br>Pay slip per employee: gross breakdown, EPF (8%), ETF (3%), PAYE tax by bracket, net pay. Batch summary: total payroll cost. |
| ⚠ CON- STRAINTS | • OT rate = 1.5× hourly (annual salary / 2080)<br>• PAYE: 0% up to 100k, 6% 100k–141k, 18% 141k–225k, 24% above |

| | |
|---|---|
| | • EPF employee: 8%, employer: 12%; ETF: 3% |
| JAVA CONCEPTS | • Static methods<br>• Progressive tax logic<br>• double[] return pattern<br>• printf() formatting<br>• Constants for rates |
| SOLUTION APPROACH | Each statutory deduction is its own method returning a double. computeGross() totals base + OT + allowances. computeTax() applies progressive brackets. printPaySlip() calls all and formats output. |
| CHALLENGE EXTENSION | Add a computeAnnualBonus() method that awards 1 month salary if employee has worked more than 5 years (add years of service as input). |

## Exercise 2.8 Hotel Room Availability Dashboard

*Scenario: Serenity Boutique Hotel*

75–90 min
**Medium**

| | |
|---|---|
| BACK- GROUND | Serenity Hotel's front desk needs a real-time room availability dashboard that shows which rooms are available, occupied, or under maintenance across all floors. |
| PROBLEM | Build a 2D-array-based room grid (4 floors × 10 rooms). Implement checkIn(), checkOut(), setMaintenance(), searchAvailableRooms(), and displayFloorPlan() static methods. |
| OBJECTIVES | • Use 2D char/String arrays for grid data<br>• Develop methods operating on 2D arrays<br>• Implement search returning count of available rooms<br>• Build a visual floor plan using loops |
| INPUT REQUIREMENTS<br>• Floor number (1–4)<br>• Room number (1–10)<br>• Guest name for check-in<br>• Maintenance flag | EXPECTED OUTPUT<br>ASCII floor plan: 'A' (Available), 'O' (Occupied), 'M' (Maintenance) per room. Check-in/out confirmation. Count of available rooms per floor. |
| CON- STRAINTS | • Cannot check in to occupied or maintenance room<br>• Cannot check out a vacant room<br>• Room status transitions: Available→Occupied→Available or Available→Maintenance→Available |
| JAVA CONCEPTS | • 2D arrays<br>• Nested for loops |

| | |
|---|---|
| | • static methods<br>• char vs String<br>• Input validation |
| SOLUTION APPROACH | Use a 2D String[][] where each cell holds status. displayFloorPlan() uses nested loops to print a formatted grid. Each action method validates the current state before transitioning. |
| CHALLENGE EXTENSION | Add occupancy statistics: occupancy rate per floor and overall hotel occupancy percentage. |

| **Exercise 2.9 Parking Slot Allocator**<br>*Scenario: SmartPark City Parking System* | ⏱ 60–75 min<br>**Easy–Medium** |
|---|---|
| BACK- GROUND | SmartPark needs an automated slot allocator to assign the nearest available parking slot to incoming vehicles, differentiating between bike bays and car parks. |
| PROBLEM | Model a parking lot with 3 zones (A=bikes: 10 slots, B=cars: 20 slots, C=large vehicles: 5 slots). Build methods to allocate(), release(), findNearest(), and printLotStatus(). |
| OBJECTIVES | • Allocate array slots sequentially<br>• Search arrays for first available slot<br>• Track allocation with parallel arrays<br>• Design methods that modify shared state |
| INPUT REQUIREMENTS<br>• Vehicle type (Bike/Car/Large)<br>• Vehicle plate number<br>• Action (Enter/Exit) | EXPECTED OUTPUT<br>Slot allocation message: 'Zone B, Slot 7 assigned to CAR-1234'. On exit: parking duration and fee. Full status board showing all zones. |
| CON- STRAINTS | • Allocate nearest (lowest index) available slot<br>• No cross-zone allocation<br>• Track entry timestamps using a counter (simple integer minutes since opening) |
| JAVA CONCEPTS | • boolean[] for slot availability<br>• String[] for plate numbers<br>• Array linear search<br>• Static methods<br>• Constants for zone sizes |
| SOLUTION APPROACH | findNearest() linearly scans the zone boolean array for the first false (empty) slot. allocate() sets it true and records the plate. release() resets it and computes fee based on duration. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add a reserveSlot() method that pre-allocates a slot for a future reservation given a plate number. |

## Exercise 2.10 Blood Bank Inventory Checker

*Scenario: Sunrise General Hospital*

60–75 min

**Medium**

| | |
|---|---|
| BACK- GROUND | The hospital blood bank needs a console tool to manage blood unit inventory by blood group (A+, A−, B+, B−, AB+, AB−, O+, O−), record donations, and check compatibility for transfusions. |
| PROBLEM | Build a method-based blood bank manager. Implement donate(), request(), checkCompatibility(), printInventory(), and findShortages() methods. Hard-code compatibility rules. |
| OBJECTIVES | • Use String arrays for blood group labels<br>• Implement lookup in parallel int[] for units<br>• Encode compatibility logic as a 2D boolean array<br>• Define meaningful domain-specific methods |
| INPUT REQUIREMENTS<br>• Action: Donate/Request/Status<br>• Blood group selection<br>• Number of units | EXPECTED OUTPUT<br>Donation confirmation with updated count. Request: compatible donor groups if requested group unavailable. Inventory table with units and shortage flag (< 5 units = shortage). |
| CON- STRAINTS | • Minimum donation: 1 unit, maximum: 5 units<br>• Request denied only if no compatible group has sufficient units<br>• Shortage threshold: < 5 units per group |
| JAVA CONCEPTS | • String arrays<br>• Parallel int[] for counts<br>• 2D boolean[][] for compatibility<br>• Array search<br>• Method decomposition |
| SOLUTION APPROACH | Encode blood groups in a String[8] array. int[8] tracks units. 8×8 boolean[][] encodes compatibility. checkCompatibility() method receives requested group index and finds any compatible available group. |
| CHALLENGE EXTENSION | Add an urgent alert: if any blood group drops to 0 units, print an urgent donation request message. |

# LEVEL 3
# INTERMEDIATE

*Object-Oriented Programming*

Classes • Objects • Constructors • Inheritance • Polymorphism • Encapsulation • Abstraction • Interfaces

## Exercise 3.1 Library Management OOP System

*Scenario: City Public Library*

⏱ 90–120 min
🎯 **Medium**

| | |
|---|---|
| **BACK- GROUND** | The library is growing and the simple array-based system can no longer scale. The IT team wants a proper OOP-based system with classes representing real-world library entities. |
| **PROBLEM** | Design and implement a class hierarchy: LibraryItem (abstract) → Book, DVD, Magazine. Implement a Member class and a Library class that manages collections of items and members. |
| **OBJECTIVES** | • Design abstract classes and concrete subclasses<br>• Apply encapsulation with private fields and getters/setters<br>• Implement method overriding (toString, display)<br>• Use ArrayList to manage dynamic collections |
| **INPUT REQUIREMENTS**<br>• Menu-driven: add item, register member, borrow, return, search | **EXPECTED OUTPUT**<br>Formatted library transactions. Overdue report. Item catalogue by type. |
| **CON- STRAINTS** | • Book: max 14-day borrow; DVD: 7 days; Magazine: not borrowable<br>• Member can hold max 3 items at once<br>• Overdue fine: LKR 10/day |
| **JAVA CONCEPTS** | • Abstract class<br>• extends<br>• Polymorphism (overriding)<br>• ArrayList<br>• Encapsulation<br>• instanceof |
| **SOLUTION APPROACH** | LibraryItem holds common fields (ID, title, available) and abstract methods (display(), getBorrowDays()). Book/DVD override them. Library uses ArrayList<LibraryItem> and ArrayList<Member>. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add an EBook subclass that tracks download count instead of physical copies. |

## Exercise 3.2 Hospital Patient Management OOP

*Scenario: Sunrise General Hospital*

120 min
**Medium**

| | |
|---|---|
| BACK- GROUND | The hospital requires a multi-class system to manage patients, doctors, and appointments in an object-oriented manner with proper role-based representation. |
| PROBLEM | Build a class hierarchy: Person (abstract) → Patient, Doctor. Create an Appointment class and a Hospital class. Implement scheduling, patient records, and prescription management. |
| OBJECTIVES | • Apply inheritance from an abstract Person class<br>• Implement interface (Schedulable) with schedule(), cancel()<br>• Demonstrate polymorphism with a Person[] array<br>• Encapsulate sensitive patient data |
| INPUT REQUIREMENTS | • Register patient/doctor, schedule appointment, add prescription, view records |
| EXPECTED OUTPUT | Patient record with appointment history. Doctor's daily schedule. Prescription list per patient. |
| CON- STRAINTS | • Doctor can have max 8 appointments/day<br>• Patient must be registered before booking<br>• Specialisation determines available appointment types |
| JAVA CONCEPTS | • Abstract class<br>• Interface<br>• ArrayList<br>• Encapsulation<br>• Polymorphism<br>• Constructor chaining (super()) |
| SOLUTION APPROACH | Person has protected common fields. Patient adds medicalHistory list. Doctor adds appointments list. Appointment holds references to both. Hospital manages master lists and scheduling rules. |
| CHALLENGE EXTENSION | Add an InsuranceProvider interface with computeCoverage() method. Implement it in the Patient class. |

## Exercise 3.3 Banking System with Accounts

*Scenario: National Savings Bank*

⏱ 120–150 min
🎯 **Medium–Hard**

| | |
|---|---|
| 📖 BACK- GROUND | The bank needs a proper OOP model for different account types with shared base behaviour and type-specific business rules for interest, minimum balance, and overdraft. |
| ❓ PROBLEM | Create an Account (abstract) class with subclasses: SavingsAccount, CurrentAccount, FixedDeposit. Implement a Customer class and a Bank class. Apply polymorphism for monthly statement generation. |
| 🎓 OBJECTIVES | • Use abstract methods for account-type-specific behaviour<br>• Apply interface Transferable for fund transfers<br>• Implement method overriding for interest calculation<br>• Demonstrate polymorphism via Account references |
| 📥 INPUT REQUIREMENTS | • Account type selection, deposit, withdrawal, transfer, statement request |
| 📤 EXPECTED OUTPUT | Monthly statement per account. Interest accrual notification. Transfer confirmation with both account balances. |
| ⚠ CON- STRAINTS | • Savings: min balance LKR 1,000, 5% annual interest; Current: overdraft limit LKR 50,000; Fixed: no withdrawal before maturity |
| 💡 JAVA CONCEPTS | • Abstract class<br>• Concrete subclasses<br>• Interface (Transferable)<br>• Polymorphism<br>• instanceof<br>• ArrayList<Account> |
| 🔧 SOLUTION APPROACH | Account is abstract with abstract calculateInterest(). Each subclass implements it differently. Bank holds ArrayList<Account>. Transfer goes through Transferable interface. Statement iterates transaction log. |
| 🚀 CHALLENGE EXTENSION | Add a LoanAccount subclass with EMI calculation and repayment schedule. |

## Exercise 3.4 E-Commerce Product Catalog

*Scenario: ShopEase Online Platform*

⏱ 90–120 min
🎯 **Medium**

| | |
|---|---|
| BACK- GROUND | ShopEase needs a properly designed OOP product catalogue with different product categories, inventory management, and a shopping cart with multiple discount strategies. |
| PROBLEM | Design Product (abstract) → Electronics, Clothing, Grocery subclasses. Create Cart, Order, and Customer classes. Implement a Discountable interface for category-specific discount logic. |
| OBJECTIVES | • Apply OOP to model an e-commerce domain<br>• Use interface for discount strategies<br>• Implement toString() and equals() overrides<br>• Manage object relationships (Cart has Products) |
| INPUT REQUIREMENTS<br>• Browse catalogue, add to cart, apply coupon, checkout | EXPECTED OUTPUT<br>Product listing with category-specific attributes. Cart summary. Order confirmation with discount breakdown. |
| CON- STRAINTS | • Electronics: warranty field; Clothing: size/colour; Grocery: expiry date<br>• Grocery discounted if near expiry<br>• Cart max: LKR 500,000 |
| JAVA CONCEPTS | • Abstract class<br>• Interface (Discountable)<br>• Method overriding<br>• Object composition<br>• ArrayList<br>• Comparable<Product> |
| SOLUTION APPROACH | Product has abstract getDescription() and double getDiscount(). Each subclass adds its own fields. Cart contains a List<Product>. Checkout creates an Order, applying interface-based discounts. |
| CHALLENGE EXTENSION | Implement Comparable<Product> on Product to sort by price and demonstrate Collections.sort(). |

## Exercise 3.5 Hotel Reservation System OOP

*Scenario: Serenity Boutique Hotel*

120–150 min

**Medium**

| | |
|---|---|
| BACK- GROUND | Serenity Hotel needs an OOP reservation system modelling rooms, guests, reservations, and services as objects with proper encapsulation and polymorphic behaviour. |

| | |
|---|---|
| PROBLEM | Create Room (abstract) → StandardRoom, DeluxeRoom, Suite. Build Guest, Reservation, and HotelService classes. Implement Billable interface for total cost computation. |
| OBJECTIVES | • Design a layered OOP class hierarchy for the hotel domain<br>• Apply encapsulation with validation in setters<br>• Use polymorphism to compute bills for different room types<br>• Manage object lifecycle: reservation creation to checkout |
| INPUT REQUIREMENTS<br>• Guest registration, room booking, add services, checkout | EXPECTED OUTPUT<br>Reservation confirmation slip. Guest bill with room charges and services. Hotel occupancy report. |
| CON- STRAINTS | • Suite: max 3 adults; Deluxe: max 2; Standard: max 2<br>• Room not double-bookable on same date<br>• Cancellation: full refund if > 48h before check-in |
| JAVA CONCEPTS | • Abstract class<br>• Interface (Billable)<br>• Object composition<br>• ArrayList<br>• Encapsulation<br>• Polymorphism |
| SOLUTION APPROACH | Room subclasses override getPricePerNight() and getMaxOccupancy(). Reservation holds Guest + Room + services list. HotelService is a separate class. Checkout calls Billable.computeTotal(). |
| CHALLENGE EXTENSION | Add a LoyaltyProgram class that tracks points (10 points per LKR 1,000 spent) and applies tier-based discounts. |

| Exercise 3.6 University Student Information System<br>*Scenario: Westbrook University* | 120 min<br>**Medium** |
|---|---|
| BACK- GROUND | The university registrar needs a fully object-oriented Student Information System to manage enrolments, results, and academic standing with proper class modelling. |
| PROBLEM | Build Person → Student, Lecturer hierarchy. Create Course, Enrolment, and Department classes. Implement GradeReport generation using polymorphism. Apply Comparable for rank ordering. |
| OBJECTIVES | • Model university domain entities as classes<br>• Implement Comparable<Student> for GPA ranking |

| | |
|---|---|
| | • Use composition: Student has a list of Enrolments<br>• Demonstrate interface segregation with Gradable interface |
| **INPUT REQUIREMENTS**<br>• Enrol student, add marks, generate transcript, show class rank | **EXPECTED OUTPUT**<br>Student transcript: courses, marks, grades, GPA. Class rank list sorted by GPA. Dean's list (GPA ≥ 3.7). |
| **⚠ CON- STRAINTS** | • Max 6 courses per student per semester<br>• GPA computed as weighted average by credit hours<br>• Student placed on probation if GPA < 2.0 |
| **JAVA CONCEPTS** | • Inheritance<br>• Interface (Comparable, Gradable)<br>• Composition<br>• Collections.sort()<br>• ArrayList |
| **SOLUTION APPROACH** | Enrolment links Student to Course with a mark. Student.calculateGPA() iterates enrolments. Comparable is on GPA. GradeReport iterates a student's enrolments and formats output. |
| **CHALLENGE EXTENSION** | Add a Scholarship class that evaluates students and auto-awards scholarships based on GPA and financial need. |

| **Exercise 3.7 Employee Payroll System OOP**<br>*Scenario: TechSoft Solutions Pvt Ltd* | ⏱ 120–150 min<br>**Medium–Hard** |
|---|---|
| **BACK- GROUND** | TechSoft needs a full OOP payroll system that handles different employee types (full-time, part-time, contract) with type-specific pay rules and a central HR management system. |
| **? PROBLEM** | Build Employee (abstract) → FullTimeEmployee, PartTimeEmployee, Contractor. Implement Taxable interface and BenefitsEligible interface. Create PayrollSystem class for batch processing. |
| **OBJECTIVES** | • Apply abstract classes for shared payroll logic<br>• Use multiple interfaces on concrete classes<br>• Demonstrate runtime polymorphism for payroll processing<br>• Build a batch-processing PayrollSystem |
| **INPUT REQUIREMENTS**<br>• Add employee, set hours/salary, run monthly payroll, generate payslips | **EXPECTED OUTPUT**<br>Individual payslips per employee type showing different deduction structures. Monthly payroll report: total cost by department. |

| | |
|---|---|
| ⚠ CON- STRAINTS | • FullTime: EPF/ETF eligible; PartTime: hourly, no benefits; Contractor: flat invoice, no deductions<br>• PAYE tax applied only to FullTime and PartTime |
| JAVA CONCEPTS | • Abstract class<br>• Multiple interfaces<br>• Polymorphism<br>• instanceof<br>• ArrayList<br>• Method overriding |
| SOLUTION APPROACH | Employee.computeGross() is abstract. Each subclass implements it per its pay model. PayrollSystem iterates Employee list, calls computeGross() polymorphically, then applies Taxable interface for deductions. |
| CHALLENGE EXTENSION | Add annual performance review: applyRaise(double percentage) in FullTimeEmployee that also recomputes EPF. |

**Exercise 3.8**

## Food Delivery App OOP

*Scenario: QuickBite Food Delivery App*

⏱ 120 min

**Medium**

| | |
|---|---|
| BACK-GROUND | QuickBite's platform must model restaurants, menus, customers, and delivery agents as proper OOP objects with clear roles and responsibilities. |
| PROBLEM | Build Restaurant, MenuItem, Customer, Order, DeliveryAgent, and OrderStatus classes. Implement Trackable interface on Order. Use enums for OrderStatus. Apply Observer pattern conceptually. |
| OBJECTIVES | • Use enums for state management<br>• Implement object composition for complex domain<br>• Apply interface for tracking functionality<br>• Model state transitions in an Order lifecycle |
| INPUT REQUIREMENTS | • Customer selects restaurant, builds order, places order, track status |
| EXPECTED OUTPUT | Order placed confirmation. Status updates: PLACED→CONFIRMED→PREPARING→OUT_FOR_DELIVERY→DELIVERED. Final receipt. |
| ⚠ CON-STRAINTS | • Order from one restaurant only<br>• Minimum order: LKR 300<br>• Agent assigned automatically from available pool |

| | |
|---|---|
| JAVA CONCEPTS | • Enum<br>• Composition<br>• Interface (Trackable)<br>• ArrayList<br>• State machine logic<br>• toString() override |
| SOLUTION APPROACH | Order holds references to Customer, Restaurant, List<MenuItem>, and DeliveryAgent. Status is an enum. updateStatus() transitions through the lifecycle. Trackable.getStatusUpdate() returns a formatted status string. |
| CHALLENGE EXTENSION | Add a RatingSystem: Customer can rate the order (1–5 stars); DeliveryAgent has a running average rating. |

### Exercise 3.9 Inventory Management OOP

*Scenario: MegaMart Inventory Control*

⏱ 90–120 min
**Medium**

| | |
|---|---|
| BACK- GROUND | MegaMart's growing inventory requires a proper OOP model for products, suppliers, warehouses, and stock movements with encapsulated business logic. |
| PROBLEM | Design Product → PerishableProduct, NonPerishableProduct. Create Supplier, Warehouse, and StockMovement classes. Implement Replenishable interface for automatic reorder logic. |
| OBJECTIVES | • Apply inheritance for product types with different expiry rules<br>• Use composition: Warehouse has Products<br>• Implement interface for reorder logic<br>• Track stock movements as objects |
| INPUT REQUIREMENTS<br>• Add product, record stock-in, stock-out, view inventory, trigger reorder | EXPECTED OUTPUT<br>Inventory report with stock levels. Expiry alerts for perishables. Automatic reorder notifications. Stock movement log. |
| CON- STRAINTS | • Perishable: expiry date mandatory, alert if < 3 days to expiry<br>• Reorder triggered when stock < minThreshold<br>• Stock movement timestamps auto-generated |
| JAVA CONCEPTS | • Inheritance<br>• Interface (Replenishable)<br>• Composition<br>• LocalDate for expiry<br>• ArrayList<br>• Encapsulation |

| | |
|---|---|
| SOLUTION APPROACH | Product base class has stock level and minThreshold. PerishableProduct adds expiryDate and overrides checkAlert() to also check expiry. StockMovement objects are created on each in/out event. |
| CHALLENGE EXTENSION | Add a SupplierPortal class with a requestQuote() method that prints a formatted purchase order. |

### Exercise 3.10 Smart Parking OOP System

*Scenario: SmartPark City Parking System*

120–150 min
**Medium–Hard**

| | |
|---|---|
| BACK- GROUND | SmartPark needs a full OOP redesign: Parking slots, vehicles, tickets, and payment are separate domain objects. Different vehicle types pay different rates with proper type hierarchy. |
| PROBLEM | Build Vehicle (abstract) → Motorcycle, Car, Van, Lorry. Create ParkingSlot, ParkingTicket, Payment, and ParkingLot classes. Implement Payable interface. Use enum for SlotStatus. |
| OBJECTIVES | • Apply full OOP hierarchy for a real system<br>• Use enum for status management<br>• Design Payable interface for payment processing<br>• Manage complex object relationships |
| INPUT REQUIREMENTS | • Vehicle entry (auto-assign slot), vehicle exit (compute fee), view lot status |
| EXPECTED OUTPUT | Entry ticket with slot assignment. Exit receipt with duration and fee. Live lot occupancy dashboard. |
| CON- STRAINTS | • Motorcycle: Zone A; Car/Van: Zone B; Lorry: Zone C<br>• Ticket must be generated on entry<br>• Payment required before exit barrier opens |
| JAVA CONCEPTS | • Abstract class<br>• Enum<br>• Interface (Payable)<br>• LocalDateTime for duration<br>• Object composition<br>• ArrayList |
| SOLUTION APPROACH | ParkingLot manages three zone lists. On entry, findAvailableSlot() scans the matching zone. ParkingTicket stores entry time. On exit, Payment is created computing duration via ChronoUnit.HOURS.between(). |
| CHALLENGE EXTENSION | Add a MonthlyPass class that grants free parking for registered vehicles, skippping the Payable interface. |

# LEVEL 4
# ADVANCED

*Exception Handling, Collections, File I/O & JDBC*

Exception Handling • Collections Framework • File I/O • Generics • JDBC • Multithreading

## Exercise 4.1 Library System with File Persistence

*Scenario: City Public Library*

⏱ 150–180 min
🎯 **Medium–Hard**

**BACK- GROUND**

The library OOP system from Level 3 loses data on restart. The team wants to add file-based persistence so all catalogue and member data survives application restarts.

**PROBLEM**

Extend the library OOP system with file I/O. Serialize books/members to CSV files on exit and deserialize on startup. Implement custom exceptions for borrow violations. Use HashMap for fast book lookup.

**OBJECTIVES**

- Implement custom exceptions (BookNotFoundException, MemberBorrowLimitException)
- Use BufferedReader/Writer for CSV I/O
- Apply HashMap<String, LibraryItem> for O(1) lookup
- Handle IOException gracefully with try-with-resources

**INPUT REQUIREMENTS**

- Application auto-loads from library_books.csv and members.csv on startup

**EXPECTED OUTPUT**

Data persisted to CSV on shutdown. Application restores previous state on next launch. Exception messages clearly indicate the problem.

**CON- STRAINTS**

- Use try-with-resources for all file I/O
- Custom exceptions must carry meaningful messages
- CSV format must be consistent and parseable

**JAVA CONCEPTS**

- Custom exceptions
- try-with-resources
- BufferedReader/Writer
- FileWriter
- HashMap
- Exception chaining

**SOLUTION APPROACH**

Create BookNotFoundException extends RuntimeException. On load, read CSV line-by-line and parse fields. On save, iterate

| | |
|---|---|
| | HashMap values and write each as a CSV row. Wrap checked IOExceptions in unchecked custom exceptions. |
| CHALLENGE EXTENSION | Add serialisation using Java ObjectOutputStream/ObjectInputStream as an alternative to CSV. Compare both approaches. |

### Exercise 4.2 Hospital Records with Exception Safety

*Scenario: Sunrise General Hospital*

⏱ 180 min
Hard

| | |
|---|---|
| BACK- GROUND | The hospital information system must be robust against invalid inputs, missing records, and concurrent access. It must log all errors to a file and maintain data integrity. |
| PROBLEM | Add exception handling and logging to the hospital OOP system. Create PatientNotFoundException, SchedulingConflictException, and InvalidVitalSignException. Implement an ErrorLogger that writes to errors.log. |
| OBJECTIVES | • Design a custom exception hierarchy<br>• Log exceptions with timestamps to a file<br>• Use TreeMap<LocalDate, List<Appointment>> for sorted schedule<br>• Implement finally blocks for resource cleanup |
| INPUT REQUIREMENTS | • All previous hospital operations now validated with exception handling |
| EXPECTED OUTPUT | Graceful error messages to console. errors.log file with timestamp and stack trace. Sorted appointment view using TreeMap. |
| CON- STRAINTS | • Never let uncaught exceptions crash the app<br>• Error log format: [TIMESTAMP] [LEVEL] [Class] - Message<br>• TreeMap ensures date-ordered schedule display |
| JAVA CONCEPTS | • Custom exception hierarchy<br>• TreeMap<br>• File logging<br>• try-catch-finally<br>• LocalDateTime.now()<br>• Checked vs unchecked exceptions |
| SOLUTION APPROACH | Create HospitalException base class, extend for each type. ErrorLogger class with a static log(Exception) method writes to BufferedWriter in append mode. Wrap all operations in try-catch, log to file and print user-friendly message. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add a retry mechanism: if scheduling fails due to conflict, automatically find the next available slot within 7 days. |

## Exercise 4.3 Bank System with Collections & Generics

*Scenario: National Savings Bank*

⏱ 180–210 min
Hard

| | |
|---|---|
| BACK- GROUND | The bank's system needs to handle thousands of accounts efficiently using the Java Collections Framework with proper generic typing for type safety and performance. |
| PROBLEM | Redesign the bank system using: HashMap<String, Account> for account lookup, LinkedList for transaction log, PriorityQueue for pending transfers, and a generic Pair<K,V> class. Implement a fraud detection check using Collections algorithms. |
| OBJECTIVES | • Use HashMap, LinkedList, PriorityQueue appropriately<br>• Create a generic utility class Pair<K,V><br>• Apply Collections.sort() with Comparator<br>• Implement fraud detection using statistical analysis on transaction list |
| INPUT REQUIREMENTS | • Full banking operations with thousands of transactions |
| EXPECTED OUTPUT | O(1) account lookup. Transfers processed in priority order. Fraud alerts when transaction amount exceeds 3× the account's historical average. |
| CON- STRAINTS | • PriorityQueue orders by transfer amount (largest first)<br>• Fraud check on every withdrawal > LKR 100,000<br>• Transaction history: last 1,000 transactions using LinkedList with a size cap |
| JAVA CONCEPTS | • HashMap<br>• LinkedList<br>• PriorityQueue<br>• Generic class<br>• Collections utility methods<br>• Comparator |
| SOLUTION APPROACH | HashMap replaces ArrayList for accounts. LinkedList as a bounded queue for transaction history. PriorityQueue with lambda Comparator for pending transfers. Generic Pair<String, Double> for fraud report output. |
| CHALLENGE EXTENSION | Implement a generic Stack<T> class and use it to support an 'undo last transaction' feature. |

## Exercise 4.4 E-Commerce Order Processing with JDBC

*Scenario: ShopEase Online Platform*

⏱ 210–240 min
**Hard**

**BACK- GROUND**

ShopEase is moving from in-memory storage to a real database. The order management system must persist orders, products, and customers to MySQL using JDBC with proper connection management.

**PROBLEM**

Implement a JDBC-backed ShopEase backend: create DAO classes for Product, Customer, and Order. Implement full CRUD operations. Use PreparedStatement for all queries. Manage connections with a simple pool.

**OBJECTIVES**

- Design DAO (Data Access Object) classes for each entity
- Use PreparedStatement to prevent SQL injection
- Implement connection pool with a fixed-size array
- Handle SQLException with meaningful recovery

**INPUT REQUIREMENTS**

- Application reads from and writes to MySQL database

**EXPECTED OUTPUT**

CRUD confirmation messages. Product search results from DB. Order history retrieved from DB. Inventory auto-decremented on order placement.

**CON- STRAINTS**

- Always use PreparedStatement, never Statement
- Connection must be closed in finally block
- Transactions: order placement must be atomic (order + inventory update)

**JAVA CONCEPTS**

- JDBC
- DAO pattern
- PreparedStatement
- ResultSet
- Transaction (commit/rollback)
- Connection management

**SOLUTION APPROACH**

Create a DBConnection class (singleton). ProductDAO, CustomerDAO, OrderDAO each open connection from pool, execute PreparedStatement, close in finally. Order placement wrapped in a transaction.

**CHALLENGE EXTENSION**

Implement a ConnectionPool class using ArrayList<Connection> that recycles connections instead of creating new ones.

## Exercise 4.5 Hotel System with Multi-threading

*Scenario: Serenity Boutique Hotel*

⏱ 180–210 min
🎯 **Hard**

| | |
|---|---|
| 📖 BACK- GROUND | Serenity Hotel's online booking platform receives hundreds of simultaneous booking requests. Without thread safety, double bookings occur. The system must handle concurrent reservations correctly. |
| ❓ PROBLEM | Add multithreading to the hotel reservation system. Simulate 10 concurrent booking threads trying to book the same limited rooms. Use synchronisation to prevent race conditions. Implement a BookingQueueProcessor thread. |
| 🎓 OBJECTIVES | • Apply synchronized blocks to room booking logic<br>• Use ReentrantLock for fine-grained control<br>• Implement a producer-consumer pattern with BlockingQueue<br>• Demonstrate thread safety with simulated concurrent access |
| 📥 INPUT REQUIREMENTS | • Simulate 10 booking threads with randomised room requests |
| 📤 EXPECTED OUTPUT | Thread-safe booking confirmations: no double-bookings. Queue processor logs processed requests. Deadlock-free execution demonstrated. |
| ⚠ CON- STRAINTS | • Use synchronized keyword or ReentrantLock (not both)<br>• BlockingQueue capacity: 20 pending requests<br>• Threads must complete within 30 seconds timeout |
| 💡 JAVA CONCEPTS | • synchronized<br>• ReentrantLock<br>• Thread<br>• Runnable<br>• BlockingQueue<br>• ExecutorService<br>• join() |
| 🔧 SOLUTION APPROACH | Room booking method is synchronized on the room object. BookingRequest added to LinkedBlockingQueue. BookingQueueProcessor is a Runnable polling the queue. ExecutorService manages the thread pool of 10 booking threads. |
| 🚀 CHALLENGE EXTENSION | Add a ReadWriteLock to allow concurrent room status reads while serialising writes. |

## Exercise 4.6 Payroll System with Generics & File I/O

⏱ 180–210 min
🎯 **Hard**

*Scenario: TechSoft Solutions Pvt Ltd*

| | |
|---|---|
| BACK- GROUND | TechSoft's payroll system needs to export payslips to files, import employee data from CSV, and use generics to create a reusable payroll processing pipeline. |
| PROBLEM | Build a generic PayrollProcessor<T extends Employee> class. Implement CSVImporter<T> and PDFExporter (text-based) classes. Process payroll from imported CSV and export payslips as .txt files. |
| OBJECTIVES | • Create generic classes with bounded type parameters<br>• Read employee data from CSV using BufferedReader<br>• Write formatted payslips to individual .txt files<br>• Chain generic processors: import → compute → export |
| INPUT REQUIREMENTS | • employees.csv file with all employee data |
| EXPECTED OUTPUT | Console: payroll summary report. File system: one payslip .txt file per employee in /payslips/ directory. |
| CON- STRAINTS | • Generic type bound: <T extends Employee><br>• CSV must be validated on import (custom CSVParseException)<br>• Create /payslips directory if not exists |
| JAVA CONCEPTS | • Generics with bounds<br>• BufferedReader<br>• BufferedWriter<br>• FileWriter<br>• Custom exceptions<br>• File class<br>• mkdir() |
| SOLUTION APPROACH | PayrollProcessor<T> has a process(List<T>) method. CSVImporter<T> reads CSV lines, splits by comma, and creates Employee subclass instances. For each employee, create a file payslips/{ID}.txt and write formatted payslip. |
| CHALLENGE EXTENSION | Add a generic Repository<T, ID> interface with findById(), findAll(), save(), and delete() methods and implement it for Employee. |

## Exercise 4.7 Food Delivery with Concurrent Order Processing

*Scenario: QuickBite Food Delivery App*

180 min

**Hard**

| | |
|---|---|
| BACK- GROUND | QuickBite receives hundreds of simultaneous orders during peak hours. Order processing, payment verification, and delivery assignment must happen concurrently without data corruption. |
| PROBLEM | Implement concurrent order processing: a main order thread accepts orders into a BlockingQueue, a PaymentProcessor thread verifies payments, and a DeliveryAssigner thread assigns agents. Use thread-safe collections throughout. |
| OBJECTIVES | • Design a multi-threaded pipeline with BlockingQueues<br>• Use ConcurrentHashMap for thread-safe order tracking<br>• Apply AtomicInteger for order ID generation<br>• Implement Callable and Future for payment verification |
| INPUT REQUIREMENTS<br>• Simulated order stream: 50 orders across 10 restaurant threads | EXPECTED OUTPUT<br>Real-time console output: orders received, payment status, agent assigned, order dispatched. Final summary: total orders processed, average processing time. |
| CON- STRAINTS | • Use ConcurrentHashMap, not HashMap, for shared order maps<br>• AtomicInteger for thread-safe order ID generation<br>• ExecutorService with 5 payment verifier threads |
| JAVA CONCEPTS | • BlockingQueue<br>• ConcurrentHashMap<br>• AtomicInteger<br>• Callable<br>• Future<br>• ExecutorService<br>• Thread pipeline |
| SOLUTION APPROACH | Three-stage pipeline: OrderReceiver → paymentQueue → PaymentProcessor → deliveryQueue → DeliveryAssigner. Each stage is a Runnable consuming from one queue and producing to the next. ConcurrentHashMap tracks order status. |
| CHALLENGE EXTENSION | Add a ScheduledExecutorService to print a live dashboard every 5 seconds showing queue sizes and completed orders. |

**Exercise 4.8**

## Inventory Management with JDBC & Transactions

*Scenario: MegaMart Inventory Control*

210–240 min

**Hard**

| | |
|---|---|
| BACK- GROUND | MegaMart's inventory system must handle concurrent stock updates (multiple cashiers scanning items) with database-backed transactions ensuring stock never goes negative. |
| PROBLEM | Build a JDBC-backed inventory system with DAO classes for Product, StockMovement, and Supplier. Implement a StockUpdateService that uses DB transactions to atomically update stock and create movement records. |
| OBJECTIVES | • Design transactional stock updates preventing negative stock<br>• Use CallableStatement for stored procedures<br>• Implement optimistic locking with version column<br>• Generate inventory reports from DB queries |
| INPUT REQUIREMENTS<br>• Stock-in from supplier (CSV import), stock-out at point of sale | EXPECTED OUTPUT<br>DB-persisted stock levels. Movement history report. Automatic reorder notification (printed) when stock < threshold. |
| CON- STRAINTS | • Transaction: stock-out must check availability, update stock, log movement atomically<br>• Optimistic lock: version column prevents lost update<br>• Stored procedure for reorder check |
| JAVA CONCEPTS | • JDBC transactions<br>• CallableStatement<br>• Optimistic locking<br>• PreparedStatement<br>• ResultSet<br>• DAO pattern |
| SOLUTION APPROACH | StockUpdateService.processStockOut() begins a transaction, checks current stock (with version), decrements, logs movement, commits. On version mismatch, rollback and retry. CallableStatement invokes checkReorder stored procedure. |
| CHALLENGE EXTENSION | Add a batch stock-in feature using PreparedStatement.addBatch() to insert hundreds of movement records efficiently. |

## Exercise 4.9 Student System with Collections Deep Dive

*Scenario: Westbrook University*

150–180 min

**Hard**

| | |
|---|---|
| BACK- GROUND | The university registrar needs advanced data management: searching students by multiple criteria, grouping by department, and generating complex reports using Java Collections advanced features. |

| | |
|---|---|
| ? PROBLEM | Redesign the student system using: TreeMap<Double, List<Student>> for GPA grouping, LinkedHashMap for insertion-ordered records, PriorityQueue<Student> for scholarship ranking. Implement advanced queries using Iterators. |
| OBJECTIVES | • Use TreeMap for sorted key access (GPA ranges)<br>• Apply LinkedHashMap to preserve insertion order<br>• Use PriorityQueue with custom Comparator for ranking<br>• Implement Iterator pattern for custom traversal |
| INPUT REQUIREMENTS<br>• All student management operations | EXPECTED OUTPUT<br>GPA-range reports using TreeMap navigation (headMap, tailMap, subMap). Scholarship candidate list from PriorityQueue. Insertion-ordered display using LinkedHashMap. |
| ⚠ CON- STRAINTS | • TreeMap keyed by GPA (Double) with List<Student> values for ties<br>• PriorityQueue Comparator: GPA descending, then name ascending<br>• Use subMap() to get students in a GPA range |
| JAVA CONCEPTS | • TreeMap<br>• LinkedHashMap<br>• PriorityQueue<br>• Comparator<br>• Iterator<br>• Collections.unmodifiableMap()<br>• subMap() |
| SOLUTION APPROACH | When a student's GPA is computed, add to TreeMap: key=GPA, value=List<Student>. For scholarship, offer PriorityQueue<Student> with lambda comparator. Use tailMap(3.5) to get dean's list students. |
| CHALLENGE EXTENSION | Use Collections.frequency() and Collections.disjoint() in a method that finds students enrolled in exactly the same set of courses. |

| | |
|---|---|
| **Exercise 4.10 Smart Parking JDBC Backend**<br>*Scenario: SmartPark City Parking System* | ⏱ 180–210 min<br>**Hard** |
| BACK- GROUND | SmartPark is deploying to multiple city locations. All parking data must be centralised in a MySQL database with real-time vehicle tracking, historical reporting, and revenue analytics. |
| ? PROBLEM | Build a full JDBC-backed SmartPark backend. DAO classes for Vehicle, ParkingSlot, Ticket, and Payment. Implement revenue |

| | |
|---|---|
| | reporting queries with GROUP BY, generate daily reports as .csv files. |
| **OBJECTIVES** | • Implement all CRUD operations via JDBC DAOs<br>• Write complex SQL queries with GROUP BY and aggregate functions<br>• Generate CSV reports using FileWriter from query results<br>• Handle concurrent ticket creation with DB-level locks |
| **INPUT REQUIREMENTS**<br>• Vehicle entry/exit events, admin requests for revenue reports | **EXPECTED OUTPUT**<br>DB-backed ticket issuance. Revenue report CSV: revenue by zone, by vehicle type, by hour. System console shows daily total. |
| **⚠ CON- STRAINTS** | • Use DB-generated IDs (AUTO_INCREMENT)<br>• Revenue query must use GROUP BY vehicle_type, zone<br>• CSV report saved to reports/daily_{date}.csv |
| **JAVA CONCEPTS** | • JDBC<br>• PreparedStatement<br>• ResultSet<br>• GROUP BY queries<br>• FileWriter for CSV<br>• Transaction<br>• DAO pattern |
| **SOLUTION APPROACH** | TicketDAO.create() inserts and retrieves generated key. PaymentDAO.getDailyRevenue() executes aggregate query. ResultSet iterated to build CSV rows. FileWriter streams rows to daily report file. |
| **CHALLENGE EXTENSION** | Implement a view layer: use ResultSetMetaData to dynamically print any query result as a formatted table. |

# LEVEL 5
# PROFESSIONAL

*Design Patterns, APIs, Testing & System Design*

Design Patterns • REST API • Unit Testing • GUI (JavaFX) • Microservices Concepts • CI/CD Awareness

## Exercise 5.1 Library System with Design Patterns

*Scenario: City Public Library*

⏱ 240–300 min
🎯 **Hard**

| | |
|---|---|
| **BACK- GROUND** | The library system is going to production. The architecture must be maintainable, extensible, and follow proven software design patterns to handle future changes without rewriting core logic. |
| **PROBLEM** | Refactor the library system applying: Singleton for LibraryDatabase, Observer for overdue notifications, Strategy for search algorithms, Factory for LibraryItem creation, and Decorator for premium membership features. |
| **OBJECTIVES** | • Implement Singleton with double-checked locking<br>• Apply Observer: overdue system notifies librarians and members<br>• Use Strategy for interchangeable search algorithms<br>• Apply Factory Method for polymorphic item creation |
| **INPUT REQUIREMENTS**<br>• All library operations, now routed through pattern-based architecture | **EXPECTED OUTPUT**<br>Pattern documentation in console: which pattern triggered. Overdue notifications on all registered observers. Pluggable search strategy demonstrated. |
| **⚠ CON- STRAINTS** | • Singleton must be thread-safe<br>• Observer list: at least LibrarianObserver and SMSObserver (simulated)<br>• Factory: separate factory for digital vs physical items |
| **JAVA CONCEPTS** | • Singleton<br>• Observer<br>• Strategy<br>• Factory Method<br>• Decorator<br>• Dependency Injection |

| | |
|---|---|
| SOLUTION APPROACH | LibraryDatabase.getInstance() with synchronized. SearchStrategy interface with TitleSearch, AuthorSearch implementations. ItemFactory.createItem(type) returns the right subclass. Decorator wraps Member adding premium features. Observer list notified on overdue. |
| CHALLENGE EXTENSION | Add a Command pattern for undo/redo of library operations (borrow, return, add item). |

**Exercise 5.2**

## Hospital System REST API Client

*Scenario: Sunrise General Hospital*

240 min

**Hard**

| | |
|---|---|
| BACK- GROUND | The hospital is integrating with the national health registry via its REST API. The Java application must call the API to verify patient NIC, retrieve allergy history, and submit new diagnoses. |
| PROBLEM | Build a REST API client using HttpClient (Java 11+). Create classes for calling: GET /patients/{nic}, POST /diagnoses, GET /allergies/{patientId}. Parse JSON responses manually using org.json or Gson. Handle API errors gracefully. |
| OBJECTIVES | • Use Java HttpClient for GET and POST requests<br>• Parse JSON responses with Gson or org.json<br>• Handle HTTP status codes (200, 400, 404, 500) with custom exceptions<br>• Build request/response model classes (DTOs) |
| INPUT REQUIREMENTS | • Patient NIC for API lookup, diagnosis data for POST |
| EXPECTED OUTPUT | Fetched patient data displayed in console. Submitted diagnosis with API-generated ID. Allergy warnings clearly highlighted. |
| CON- STRAINTS | • All API calls must have 5-second timeout<br>• 400/404 responses throw APIClientException<br>• Retry once on 500 errors |
| JAVA CONCEPTS | • HttpClient<br>• HttpRequest<br>• HttpResponse<br>• Gson / org.json<br>• DTO classes<br>• CompletableFuture for async |
| SOLUTION APPROACH | Create APIClient with methods wrapping each endpoint. DTOs: PatientDTO, DiagnosisDTO, AllergyDTO. Gson.fromJson() deserialises responses. APIClientException wraps non-2xx |

| | |
|---|---|
| | responses. Optional async using sendAsync() returning CompletableFuture. |
| CHALLENGE EXTENSION | Add async parallel API calls: fetch patient data and allergy history simultaneously using CompletableFuture.allOf(). |

| Exercise 5.3 Banking System Full Unit Test Suite<br>*Scenario: National Savings Bank* | ⏱ 240–300 min<br>Hard |
|---|---|
| BACK- GROUND | The bank's development team is adopting TDD. The entire banking system needs a comprehensive JUnit 5 test suite with mocked dependencies, edge case coverage, and test reports. |
| PROBLEM | Write a full JUnit 5 + Mockito test suite for the banking system. Cover: account creation, deposit/withdrawal edge cases, transfer logic, interest calculation, and fraud detection. Achieve > 85% code coverage. |
| OBJECTIVES | • Write JUnit 5 tests with @Test, @BeforeEach, @AfterEach, @ParameterizedTest<br>• Mock DAO/database dependencies with Mockito (@Mock, @InjectMocks)<br>• Use assertThrows() for exception testing<br>• Generate and interpret JaCoCo coverage reports |
| INPUT REQUIREMENTS<br>• No user input – test data defined in test classes | EXPECTED OUTPUT<br>JUnit console report: tests passed/failed/skipped. JaCoCo HTML coverage report showing uncovered branches. |
| CON- STRAINTS | • > 85% line coverage required<br>• Every public method has at least 3 test cases (happy path, edge case, error case)<br>• No test should depend on external DB – use mocks |
| JAVA CONCEPTS | • JUnit 5<br>• Mockito<br>• @Mock, @InjectMocks, when().thenReturn()<br>• assertThrows()<br>• @ParameterizedTest<br>• JaCoCo |
| SOLUTION APPROACH | AccountTest covers createAccount, deposit, withdraw with boundary values. TransferTest mocks AccountDAO. FraudDetectionTest parameterised with amounts triggering and not triggering fraud. @BeforeEach initialises test data. Mockito stubs DB calls. |

| | |
|---|---|
| **CHALLENGE EXTENSION** | Implement a test fixture factory class TestDataFactory that provides consistent test objects across all test classes. |

## Exercise 5.4 E-Commerce JavaFX Dashboard

*Scenario: ShopEase Online Platform*

300–360 min

**Hard**

| | |
|---|---|
| **BACK- GROUND** | ShopEase's management team wants a GUI dashboard to monitor live sales, manage products, and view order analytics without using the command line. |
| **PROBLEM** | Build a JavaFX dashboard application with multiple scenes: Login Screen, Product Management (CRUD table), Order Monitor (live-updating list), and a Sales Analytics chart using JavaFX Chart API. |
| **OBJECTIVES** | • Design a multi-scene JavaFX application with SceneBuilder or code<br>• Implement TableView<Product> with sorting and filtering<br>• Integrate JavaFX Charts (BarChart for revenue by category)<br>• Apply MVC pattern: FXML Controllers, Model classes, Service layer |
| **INPUT REQUIREMENTS**<br>• GUI interactions: login, CRUD products, view orders | **EXPECTED OUTPUT**<br>Running JavaFX application: functional login, product table with search, order monitor, revenue bar chart. |
| **CON- STRAINTS** | • Use FXML for UI layout<br>• Apply CSS styling for professional look<br>• Background thread (Task<>) for data loading to keep UI responsive |
| **JAVA CONCEPTS** | • JavaFX<br>• FXML<br>• TableView<br>• ObservableList<br>• BarChart<br>• Task<><br>• MVC with Controllers<br>• CSS |
| **SOLUTION APPROACH** | Application class switches scenes. FXMLLoader loads FXML. ProductController binds TableView to ObservableList<Product>. SearchField filters list using filtered list wrapper. BarChart bound to CategoryRevenueService data. Task<> loads data off UI thread. |

| | |
|---|---|
| CHALLENGE EXTENSION | Add real-time order notifications using a background ScheduledTask that polls for new orders every 10 seconds and adds them to the order list. |

## Exercise 5.5 Hotel System with Builder & Repository Pattern

*Scenario: Serenity Boutique Hotel*

240–300 min

**Hard**

| | |
|---|---|
| BACK- GROUND | The hotel's complex reservation object has too many constructor parameters. The system also needs a clean data access layer separating business logic from database operations. |
| PROBLEM | Refactor the hotel system using Builder pattern for Reservation creation, Repository pattern for data access, and Chain of Responsibility for booking validation (availability → payment → confirmation). |
| OBJECTIVES | • Implement Builder pattern for complex Reservation construction<br>• Build a generic Repository<T, ID> with JDBC implementation<br>• Apply Chain of Responsibility for validation pipeline<br>• Use Optional<T> for null-safe lookups |
| INPUT REQUIREMENTS | • Booking request: guest details, room type, dates, services |
| EXPECTED OUTPUT | Fluent builder API for reservations. Clean validation chain output. Optional-safe room retrieval. Repository persists to DB. |
| CON- STRAINTS | • Builder must enforce mandatory fields (guest, dates, room)<br>• Chain must stop and report the first failing validation<br>• Repository.findById() returns Optional<T> |
| JAVA CONCEPTS | • Builder pattern<br>• Repository pattern<br>• Chain of Responsibility<br>• Optional<T><br>• JDBC<br>• Generics |
| SOLUTION APPROACH | Reservation.Builder inner class with fluent setters, build() validates mandatory fields. Validator abstract class with setNext(). AvailabilityValidator → PaymentValidator → ConfirmationHandler. Repository interface with JDBC implementation returning Optional from findById(). |
| CHALLENGE EXTENSION | Add Prototype pattern: clone() an existing Reservation to make a repeat booking for returning guests. |

## Exercise 5.6 Payroll Microservice with REST Endpoint

*Scenario: TechSoft Solutions Pvt Ltd*

⏱ 300–360 min
🎯 **Hard**

| | |
|---|---|
| 📖 BACK- GROUND | TechSoft is modernising to a microservices architecture. The payroll module will be a standalone Java service exposing REST endpoints consumed by the HR portal. |
| ❓ PROBLEM | Build a simple REST API server using Java's built-in HttpServer (com.sun.net.httpserver). Expose endpoints: GET /employees, GET /employees/{id}, POST /employees, POST /payroll/run/{month}. Return JSON responses. |
| 🎓 OBJECTIVES | • Use Java HttpServer to create a REST API without a framework<br>• Parse request bodies as JSON using Gson<br>• Implement routing with HttpHandler for each endpoint<br>• Return appropriate HTTP status codes and JSON responses |
| 📥 INPUT REQUIREMENTS | • HTTP requests from Postman or curl (demonstrated in code comments) |
| 📤 EXPECTED OUTPUT | Running HTTP server on port 8080. GET /employees returns JSON array. POST creates employee. POST /payroll/run/{month} triggers payroll and returns payslip JSON. |
| ⚠ CON- STRAINTS | • All responses: Content-Type: application/json<br>• POST body validated before processing<br>• Use ThreadPool ExecutorService for the server |
| 💡 JAVA CONCEPTS | • com.sun.net.httpserver.HttpServer<br>• HttpHandler<br>• HttpExchange<br>• Gson serialisation<br>• ExecutorService<br>• HTTP methods and status codes |
| 🔧 SOLUTION APPROACH | HttpServer.create(port). Register contexts: /employees → EmployeeHandler, /payroll → PayrollHandler. Each handler checks request method (GET/POST), parses query params or body, calls service layer, writes JSON response. |
| 🚀 CHALLENGE EXTENSION | Add basic authentication: check Authorization header for Base64-encoded credentials before serving any request. |

## Exercise 5.7 Food Delivery System

⏱ 300 min
🎯 **Hard**

## Design with Patterns

*Scenario: QuickBite Food Delivery App*

| | |
|---|---|
| BACK- GROUND | QuickBite is scaling to 100,000 daily orders. The system architecture needs design patterns to handle surge pricing, notification routing, and extensible payment methods. |
| PROBLEM | Apply: Strategy for surge pricing algorithms (TimeBasedSurge, WeatherBasedSurge), Template Method for order processing pipeline, Proxy for payment gateway (with logging and retry), and Facade for the complex ordering subsystem. |
| OBJECTIVES | • Apply Strategy for interchangeable pricing algorithms<br>• Use Template Method to standardise order lifecycle hooks<br>• Implement Proxy for transparent payment logging and retry<br>• Create Facade simplifying the ordering subsystem API |
| INPUT REQUIREMENTS<br>• Order placement through simplified Facade API | EXPECTED OUTPUT<br>Surge pricing applied based on selected strategy. Order lifecycle hooks printed at each stage. Payment proxy logs and retries failed payments. Client code only calls OrderFacade.placeOrder(). |
| CON- STRAINTS | • Strategy changeable at runtime (setter injection)<br>• Proxy must be transparent (same interface as real payment gateway)<br>• Template Method: abstract hooks at least 4 stages |
| JAVA CONCEPTS | • Strategy<br>• Template Method<br>• Proxy<br>• Facade<br>• Interface segregation<br>• Logging |
| SOLUTION APPROACH | PricingStrategy interface: computePrice(order). AbstractOrderProcessor defines processOrder() calling abstract hooks: validateOrder(), chargePayment(), notifyRestaurant(), assignDelivery(). PaymentGatewayProxy wraps real gateway, logs, retries on failure. OrderFacade hides all subsystems. |
| CHALLENGE EXTENSION | Add Flyweight pattern to cache MenuItem objects, sharing immutable data across thousands of order items. |

## Exercise 5.8 Inventory System with Full Test Coverage

*Scenario: MegaMart Inventory Control*

240–300 min

**Hard**

| | |
|---|---|
| BACK- GROUND | MegaMart's inventory system is business-critical. A full TDD approach is needed: write tests first, then implement the functionality for the entire inventory management domain. |
| PROBLEM | Using strict TDD (Red → Green → Refactor), build the inventory system test-first. Write tests for: stock update (in/out), reorder trigger, expiry alert, supplier order generation, and concurrent stock updates. Use Mockito to mock the DB layer. |
| OBJECTIVES | • Practice TDD workflow: write failing test → make it pass → refactor<br>• Mock DAO layer with Mockito for unit-isolated tests<br>• Test concurrent stock update safety with threads<br>• Achieve 90%+ test coverage with JaCoCo |
| INPUT REQUIREMENTS<br>• Test-driven development – inputs defined in @ParameterizedTest and @Test methods | EXPECTED OUTPUT<br>All tests green. TDD commit log (described in README). JaCoCo report showing > 90% coverage. Mutation testing report (PIT) identifying surviving mutants. |
| CON- STRAINTS | • Write test before implementation (strict TDD)<br>• No direct DB access in unit tests (all mocked)<br>• Parameterized tests for boundary conditions |
| JAVA CONCEPTS | • JUnit 5 TDD<br>• Mockito<br>• @ParameterizedTest<br>• @MethodSource<br>• PIT mutation testing<br>• JaCoCo<br>• Test doubles |
| SOLUTION APPROACH | Start with InventoryServiceTest. Write stockOut_insufficientQuantity_throwsException() – it fails. Implement StockService.stockOut() to make it pass. Add expiry check test, implement alert logic. Each cycle Red→Green→Refactor. Mockito stubs InventoryDAO. |
| CHALLENGE EXTENSION | Use PIT (mutation testing) to identify weak tests and strengthen them until mutation score exceeds 80%. |

## Exercise 5.9 Smart Parking JavaFX with Live Map

*Scenario: SmartPark City Parking System*

360 min

**Hard**

| | |
|---|---|
| BACK- GROUND | SmartPark wants a control room application where operators can see the entire parking lot on screen, respond to incidents, and process vehicle entries/exits through a modern GUI. |
| PROBLEM | Build a JavaFX Smart Parking control panel with a live slot grid (canvas or GridPane), vehicle entry/exit processing form, revenue chart, and an alert panel. Use background threads for simulated slot updates. |
| OBJECTIVES | • Build a complex JavaFX layout with GridPane slot visualisation<br>• Use Canvas API for custom parking lot drawing<br>• Apply Task<Void> for background simulation thread<br>• Integrate charts and tables in a single BorderPane layout |
| INPUT REQUIREMENTS<br>• GUI: enter vehicle plate → assign slot; click slot → release; filter by zone | EXPECTED OUTPUT<br>Visual parking lot grid updating in real time. Revenue BarChart updating on each payment. Alert panel showing availability warnings. Export slot report to CSV button. |
| CON- STRAINTS | • UI updates only on JavaFX Application Thread (Platform.runLater)<br>• Simulation thread runs every 2 seconds, adding/removing random vehicles<br>• All slot operations thread-safe (synchronized on slot object) |
| JAVA CONCEPTS | • JavaFX Canvas<br>• GridPane<br>• Task<><br>• Platform.runLater()<br>• BarChart<br>• TableView<br>• FileChooser<br>• CSS |
| SOLUTION APPROACH | Draw parking grid on Canvas using GraphicsContext. Colour slots by status. Background Task simulates arrivals/departures, calls Platform.runLater() to update canvas. Entry form calls ParkingService, updates chart data. Export button opens FileChooser and writes CSV. |
| CHALLENGE EXTENSION | Add an animation: when a car enters, animate a car icon moving to the assigned slot using TranslateTransition. |

**Exercise 5.10 Full-Stack Student Portal REST + JavaFX**

*Scenario: Westbrook University*

360–480 min

**Hard**

| | |
|---|---|
| BACK- GROUND | Westbrook University is building a student self-service portal: a REST backend exposing student, course, and grade data, consumed by a JavaFX desktop client application. This capstone exercise integrates all previous concepts. |
| PROBLEM | Build a two-tier system: (1) REST API server (Java HttpServer + JDBC + MySQL) exposing student data endpoints. (2) JavaFX client consuming the API (HttpClient + Gson). Apply Singleton, Repository, and MVC patterns across both tiers. |
| OBJECTIVES | • Build a complete REST API server in pure Java<br>• Consume the API from a JavaFX client using HttpClient<br>• Apply layered architecture: Controller → Service → Repository → DB<br>• Integrate unit tests for service layer with mocked repositories |
| INPUT REQUIREMENTS<br>• JavaFX GUI: login, view courses, check grades, submit assignments (simulated) | EXPECTED OUTPUT<br>Running REST server on port 8080. JavaFX client fetches and displays real data. Login validates against DB. Grades displayed in TableView. JUnit tests for service layer pass. |
| CON- STRAINTS | • Strict MVC in JavaFX: no business logic in controllers<br>• All API calls from JavaFX must be on background thread<br>• Minimum 5 REST endpoints, all returning JSON |
| JAVA CONCEPTS | • HttpServer (backend)<br>• HttpClient (frontend)<br>• JDBC + MySQL<br>• Gson<br>• JavaFX + FXML<br>• Repository pattern<br>• JUnit 5<br>• MVC |
| SOLUTION APPROACH | Backend: StudentAPI registers /api/students, /api/courses, /api/grades. Each handler calls service → repository → DB. Frontend: LoginController calls authService (HttpClient). DashboardController loads data in Task<>, binds to TableView. Tests mock StudentRepository. |
| CHALLENGE EXTENSION | Add WebSocket simulation using a background server thread pushing grade change notifications to all connected JavaFX clients using a shared observer list. |